COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Programming languages, compilers and computability

Module 15 / expanded Volume 03

Four lessons, worked examples, full practical requirements and 20 lesson questions.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

15.1 — Language paradigms and type systems 3

15.2 — Lexing, parsing and syntax trees 7

15.3 — Bytecode, virtual machines and bounded execution..... 10

15.4 — Automata, computability and compiler optimization 13

Continuity with the original program..... 16

Primary reference directory..... 16

Reading by topic 18

Module 15 — Programming languages, compilers and computability

Prerequisites: Modules 11-14 and discrete proofs.

Project: Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

Primary references: R14, R06, R11. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

15.1 — Language paradigms and type systems

Learning outcome: Separate language semantics from syntax and implementation habits.

Paradigms describe how a program is organized

Imperative code updates state through a sequence of operations. Functional style emphasizes computing results from explicit inputs, often using immutable values. Object-oriented design groups behavior behind interfaces and ownership boundaries. These are compatible design tools, not mutually exclusive team identities.

A combat loop can imperatively advance time and emit effects while calling a pure damage function. The pure function can be tested with a simple input/output table because it has no hidden dependency on a global clock, random generator or mutable enemy registry. The surrounding loop still needs tests for order and effects.

Purity does not mean every implementation is allocation-free or fast. It means the chosen function's observable result follows its explicit inputs without the forbidden hidden effects in its contract. A game can use a functional core and an imperative shell while preserving native resource ownership.

Type rules reject some mistakes, not all mistakes

A static type checker applies rules before execution; a dynamic check applies at runtime. The distinction is not simply compiled versus interpreted: an interpreter can perform static checks before evaluating, and a compiled program can contain dynamic type checks.

The toy checker has two types, Number and Boolean. Addition requires two Numbers and returns Number. Logical AND requires two Booleans and returns Boolean. Reject Number + Boolean even if a host language would permit some implicit conversion. These are the toy language's rules, not an obligation to copy every C++ conversion.

A well-typed expression can still divide by zero, exceed an application limit or calculate the wrong game balance. A type system proves only what its formal rules establish. Keep type errors separate from syntax errors, runtime domain failures and gameplay requirement failures. [R14: frontend concepts; R11: I]

An ABI boundary needs a smaller vocabulary

A C-compatible interface should use types and declarations understood by a C compiler, with explicit sizes or opaque handles where appropriate. Do not expose `std::string`, references, templates or C++ exceptions across a C header. `extern "C"` controls language linkage when the header is compiled as C++; it does not turn arbitrary C++ data structures into a portable binary protocol.

Specify which side allocates, which side frees, how long borrowed buffers remain valid, whether null is permitted and how errors are returned. A status code plus an output parameter is one possible C design. On failure, leave the output unchanged or set it to a specified safe value; choose one and test it.

Binary compatibility also depends on compiler ABI, platform, calling convention, alignment and build configuration. A shared C header reduces friction but is not a promise that any binary built by any compiler can link with any other. The mandatory C bridge compiles the implementation as C17 and exercises it from C++20 on the same toolchain. [P4-05]

Keep boundary failures inside the host contract

A C++ wrapper can turn a C handle into a unique owner with a custom deleter. The deleter must call the C library's matching destroy function, not a guessed `delete` or unrelated allocator. A failed creation returns the documented status and no owned handle.

If C code calls back into C++, the callback must prevent exceptions from escaping through a boundary whose contract does not support them. Convert failures into the agreed status channel. This lesson's required interface has no callbacks, keeping that risk out of the basic exercise while making the principle explicit.

The hardware path will later add fixed register layouts, interrupts, DMA and platform-specific calling conventions. This mock interface does not access real hardware. It teaches the allocation, representation and error obligations that must be understood before device integration.

One rule, two styles and a type boundary

Rule: subtract armor from incoming damage, but never apply negative damage. With incoming 12 and armor 5, effective damage is 7. An imperative version assigns a temporary, clamps it and subtracts it from hit points. A pure version returns the new hit-point value from explicit inputs. For hit points 20, both produce 13.

The toy checker accepts Number + Number and rejects Number + Boolean. A division expression can still be type-correct with a zero denominator, so runtime domain checks remain necessary.

At the C boundary, a function `ge_apply_damage(int hp, int damage, int* out_hp)` returns a status, rejects negative domain inputs and a null output pointer, and writes only on success. Its C++ wrapper can return an optional value or throw at a C++ boundary after interpreting the status.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
#ifdef __cplusplus
extern "C" {
#endif
int ge_apply_damage(int hp, int damage, int *out_hp);
#ifdef __cplusplus
}
#endif
```

Common failure modes

Static typing does not prove all gameplay requirements. Interpreted does not imply dynamically typed. `extern "C"` does not make C++ containers C-compatible. Allocation and release must use the same API family.

Practical assignment

Original requirement: Write one rule in imperative and pure-functional styles, then define a small typed C-compatible interface for a mock subsystem. Pass: state allocation ownership and reject type-invalid expressions in a toy checker.

1. Implement the same bounded damage rule in imperative and pure-functional styles and compare a full input table.
2. Implement Number/Boolean type rules for arithmetic and logical operators.
3. Write a C-compatible header with explicit ownership, null and error policies.
4. Use the later C bridge to compile an actual C implementation and call it through a C++ wrapper.

Required cases: damage below armor -> zero effective damage; damage exceeds remaining HP -> clamp at zero; negative domain input rejected; Number+Number accepted; Number+Boolean rejected; null output rejected without dereference.

Evidence to submit: Two equivalent rule implementations, type-rule table, invalid-expression tests and a documented C boundary.

Independent transfer: A damage function reads a global random generator. Redesign its interface so a test can reproduce the same result without relying on global state.

Assigned primary reading: R14: language frontend concepts; R11: interfaces and C-style boundary guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does static typing prove all logical requirements?
2. What advantage does a pure calculation offer tests?
3. What must a C-compatible ownership boundary specify?
4. Can an interpreter perform static type checking before execution?

5. Why must a C++ wrapper call the C API's destroy function rather than `delete`?

15.2 — Lexing, parsing and syntax trees

Learning outcome: Convert source text into a structured representation with diagnostics.

Write the language specification first

The bundle's tiny expression language uses bounded signed integer values from -1,000,000,000 through +1,000,000,000. Source integer literals are unsigned decimal digits; unary minus introduces negative values. It supports parentheses and +, -, *, /. Division truncates toward zero. Every intermediate result must remain in range, and division by zero is an error.

Only ASCII whitespace and the declared token characters are accepted. Names are not part of the executable baseline: alphabetic input receives an unknown-name/invalid-token diagnostic. The Number/Boolean checker from Lesson 15.1 is a separate type-system exercise, not a claim that booleans are executable in this arithmetic grammar.

Resource limits are part of the language service contract: at most 16,384 source bytes, 4,096 tokens, 4,096 AST nodes and structural/nesting depth 64 in the reference. These are teaching limits chosen to keep failures manageable, not validated production security limits. Do not use host-language `eval` to execute the input. [R14: lexer and parser; P4-06]

Lexing groups characters; parsing gives structure

The lexer scans left to right and produces Number, Plus, Minus, Star, Slash, LeftParen, RightParen and End tokens. Each token retains a source offset. A sequence of digits forms one number; check the literal limit while accumulating or use a checked conversion. Whitespace is skipped without losing the offsets in the original source.

The parser consumes tokens according to precedence. `expression` handles addition and subtraction, `term` handles multiplication and division, and `primary` handles numbers, parentheses and unary minus. A loop for repeated addition/subtraction makes binary subtraction left-associative. A recursive call for unary minus allows a bounded chain such as `--3`.

Require End after parsing the expression. Accepting a valid prefix and ignoring trailing tokens would let `2 3` appear successful. Report the unexpected token's offset and a useful category. A syntax error is different from a runtime error such as division by zero in a syntactically valid expression.

An arena AST makes ownership visible

An abstract syntax tree stores semantic structure rather than every punctuation mark. A binary node contains an operator, child indices and the operator's source location. A literal node contains its value and location. The reference stores nodes in an owning vector and uses indices, avoiding pointers that would dangle when the vector grows.

Child indices must refer to valid earlier nodes in this constructed tree. Structural depth is computed when a node is created and checked against the limit. A long left-associative chain can have shallow parenthesis nesting but a deep AST, so checking parentheses alone is not enough. The arena's destruction is nonrecursive because its nodes own indices, not nested child owners.

Do not keep a reference to a vector element while appending another AST node. Retrieve by index after mutation. This is a direct application of the native lifetime lessons, now inside a compiler frontend.

Evaluation follows the tree and the specified arithmetic

Evaluate the left child, then the right child, then apply the node's operation. This explicit order defines which failure is observed first when both sides fail. The reference uses 64-bit intermediates for values bounded to one billion: multiplying two valid operands stays within the signed 64-bit range before checking the tighter language range.

A failed operation reports the operator's source offset. The interpreter returns one value only after the entire tree succeeds. It never partially updates the simulation during expression evaluation. Host changes happen only after the caller receives and validates a result.

Differential tests later compare tree evaluation with bytecode execution, but both share the declared arithmetic policy. Add independently calculated golden results and domain-error examples so a shared arithmetic mistake is not hidden by agreement between the two engines.

Parse and evaluate `2 + 3 * 4`

Tokens are Number(2), Plus, Number(3), Star, Number(4), End. The term parser groups 3*4 before the outer expression creates an addition node. The AST is Add(Literal(2), Mul(Literal(3),Literal(4))). Evaluation computes 3*4 = 12, then 2+12 = 14.

For `8 - 3 - 1`, the expression loop creates Sub(Sub(8,3),1), yielding 4. Right association would yield 6 and violate the specification. For `8 / 3`, the result is 2; for `-8 / 3`, it is -2.

For `1 / (2 - 2)`, syntax is valid but evaluation fails at the slash. For `2 3`, parsing fails because an extra Number remains before End. For `speed + 1`, the lexer reports the unsupported name rather than invoking a host variable.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
expression := term { ("+" | "-") term }
term       := primary { ("*" | "/") primary }
primary    := integer | "-" primary | "(" expression ")"
integer    := digit { digit }
digit      := "0" | "1" | ... | "9"
```

Common failure modes

Checking only nesting depth misses a long left-deep AST. Ignoring trailing tokens accepts malformed programs. A parser must not execute host text. An unknown name is not automatically zero. Shared evaluator agreement is not an independent arithmetic proof.

Practical assignment

Original requirement: Implement tokens, recursive-descent parsing and evaluation for numbers, parentheses and four arithmetic operators. Pass: test precedence, associativity, malformed input, unknown names and division-by-zero policy.

1. Write the grammar, numeric domain, evaluation order and error-location policy before coding.
2. Implement a bounded lexer retaining offsets and a recursive-descent parser producing an arena AST.
3. Implement evaluation with intermediate-range and division checks.
4. Test precedence, associativity, malformed input, unknown names and all declared resource limits.

Required cases: 2+3*4 -> *14; (2+3)*4 -> 20; 8-3-1 -> 4; -8/3 -> -2; 1/(2-2) -> division error at slash; empty/2 3/unknown name -> diagnostic; excess source, tokens, nodes or depth rejected.

Evidence to submit: Language specification, token/AST traces, golden-result table, located diagnostics and bounded-input tests.

Independent transfer: Add a remainder operator. Specify precedence, negative-operand behavior and zero-divisor policy before changing the lexer or parser.

Assigned primary reading: R14: Kaleidoscope lexer, parser and abstract syntax tree chapters.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What separates lexing from parsing?
2. Under ordinary precedence, what is 2+3*4?
3. Why preserve source locations in a syntax tree?
4. Why must the parser require End after the expression?
5. Why can an arena of indexed nodes be easier to own than pointers into a growable node vector?

15.3 — Bytecode, virtual machines and bounded execution

Learning outcome: Implement a small execution model with explicit limits.

A virtual machine is a specified execution model

The reference instruction set contains PUSH literal, ADD, SUB, MUL, DIV and NEG. PUSH adds one validated value to the stack. A binary arithmetic instruction consumes two operands and pushes one result; NEG consumes one and pushes one. The program has no jumps, host calls, files or network access.

Compile the AST in postorder: emit the left subtree, then the right subtree, then the operator. This order matches the tree evaluator's left-before-right error semantics. Instructions retain the source offset of the corresponding operation so runtime failures point back to the expression.

The code is an in-memory teaching representation, not a portable bytecode file format. An external byte format would need specified encoding, versioning, length validation and a decoder. Do not serialize the raw instruction structure and call it a cross-platform format. [R14: code generation concepts]

Operand order is easy to reverse accidentally

For binary subtraction, pop the right operand first and the left operand second, then compute left minus right. The top of stack contains the most recently compiled operand, not necessarily the mathematical left side. Addition can hide a reversal because it is commutative on ordinary valid integers; subtraction and division expose it.

Before popping, require at least two values. Before PUSH, require available stack capacity and a valid literal. Reject an unknown opcode through an explicit default case. At program end, require exactly one stack value. Empty code or two uncombined literals are invalid completed expressions under this VM contract.

The reference's stack has a configured maximum of 256 values. The compiler-produced expressions also obey the AST bounds, but the VM must not trust arbitrary instruction sequences merely because the normal compiler emits valid ones.

Bound every stage, not only instruction count

The VM accepts an execution budget and decrements it before each instruction. A nonempty program with budget zero fails before executing its first instruction. The reference also caps program length and stack size, and validates arithmetic at every operation.

Instruction fuel limits executed steps, not arbitrary wall-clock time. The current straight-line instructions are deliberately small and contain no unbounded host work. Adding a host callback, allocation-heavy instruction or loop changes the cost model and requires renewed limits. Parsing, lexing and AST construction already have their own limits because VM fuel cannot protect work performed before execution.

A bounded interpreter is not automatically a secure sandbox. Native implementation defects, unsafe host capabilities and environment exposure remain concerns. Keep this classroom VM local and capability-free. Production mod execution requires a separate security architecture and review. [P4-03; P4-04]

Compare engines without overstating independence

For every valid expression, compile it and compare VM output with tree evaluation. Check known results such as 2+3*4 = 14 independently. For failures, compare error category and source location where the language promises equivalence.

The reference tree evaluator and VM intentionally share arithmetic validation to avoid divergent definitions of division and range checks. Consequently their agreement is strong evidence about tree traversal, emission order and stack behavior, but weaker evidence about that shared arithmetic function. Golden examples and boundary tests address the shared dependency.

Add malformed bytecode tests that cannot come from the normal compiler: invalid opcode, underflow, out-of-range literal, excess stack, extra final values and insufficient fuel. Testing only compiler-generated bytecode leaves a large part of the VM's input contract unexplored.

Execute `8 - 3 * 2`

Bytecode is PUSH 8, PUSH 3, PUSH 2, MUL, SUB. Stack traces are [8], [8,3], [8,3,2], [8,6], [2]. MUL changes stack height by -1, as does SUB. For SUB, right = 6 and left = 8; reversing them would incorrectly produce -2.

The five-instruction program succeeds with budget 5 and fails with budget 4 before the final SUB. An isolated ADD fails before reading nonexistent operands. PUSH 1 followed by PUSH 2 fails at completion because two values remain.

Preserve the slash offset for a division instruction so `1/(2-2)` reports the same operation location as the tree evaluator.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
if (stack.size() < 2)
    throw LangError(instruction.pos, "stack underflow");
const auto right = stack.back(); stack.pop_back();
const auto left  = stack.back(); stack.pop_back();
stack.push_back(apply(instruction.op, left, right,
                      instruction.pos));
```

Common failure modes

Subtraction and division reveal reversed pops. Instruction fuel does not bound frontend work or arbitrary host callbacks. A bytecode vector is not a secure serialized format. Shared arithmetic means two evaluators are not completely independent oracles.

Practical assignment

Original requirement: Compile the expression language into stack bytecode and compare results to the tree evaluator. Pass: reject stack underflow, invalid opcodes and over-budget execution; preserve source context for errors.

1. Specify each opcode, stack effect, operand order and error location.
2. Compile the AST in postorder and compare golden stack traces.
3. Implement VM checks for malformed instructions and resource limits.
4. Differentially test valid expressions and error behavior, explicitly documenting shared validation code.

Required cases: 8-3*2 -> 2; five instructions pass with fuel 5, fail with 4; ADD on empty -> underflow; unknown opcode rejected; two final values rejected; PUSH beyond stack cap rejected; invalid literal and arithmetic range rejected.

Evidence to submit: Opcode table, stack traces, compiler/VM differential tests, malformed-bytecode tests and limit matrix.

Independent transfer: Add a conditional jump. Identify the new obligations for instruction-pointer validation, fuel, reachable stack states and source-level termination claims.

Assigned primary reading: R14: code generation and execution concepts; course security requirements.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What is the stack-height change of a binary add that pushes one result?
2. Why compare VM output to the tree evaluator?
3. Does using bytecode automatically create a secure sandbox?
4. Why do subtraction tests matter even when addition tests pass?
5. What does comparing two engines that share arithmetic validation fail to establish independently?

15.4 — Automata, computability and compiler optimization

Learning outcome: Know what analysis can establish and when transformation is legal.

Finite automata recognize bounded-memory patterns

A finite-state machine has a finite set of states, an initial state, transition rules and accepting states. A decimal-literal recognizer can use Start, Digits and Dead. From Start a digit enters Digits; from Digits another digit stays there; any other character enters Dead. Accept only if the final state is Digits.

The empty string is rejected because no digit was read. The recognizer accepts leading zeros under the stated grammar. It recognizes the digit pattern but does not by itself check the numeric value limit; an enormous decimal string can be lexically valid and numerically out of range.

A lexer uses related state transitions to classify tokens, but the complete expression grammar also has arbitrarily nested parentheses in its mathematical form. The teaching implementation imposes a depth limit. Keep lexical recognition, grammatical structure and implementation bounds distinct. [R06: computation models; R14: frontend]

Undecidability does not prohibit useful proofs

The halting problem says there is no general algorithm deciding termination for every program in a sufficiently expressive unrestricted computation model. It does not say that no particular program can be proven to terminate or that useful restricted analyses are impossible.

Our accepted expression AST is finite and acyclic. The tree evaluator visits a bounded number of nodes, and the compiled bytecode is straight-line with no jumps. Each operation is bounded by the chosen native integer implementation. These structural restrictions let us argue termination for accepted programs without solving a general halting problem.

A parser can still fail to terminate because of an implementation bug, such as a loop that never advances the token position. Prove progress of each loop and a decreasing measure or explicit bound for each recursion. A fuel counter is a runtime limit, not a proof that the source language's unrestricted future extensions always terminate.

Optimization must preserve the chosen observations

Constant folding replaces an operation on constant values with its result when doing so preserves the language's semantics. For this language, semantics include bounded integer arithmetic, left-before-right evaluation, error category and source offset. Folding 2+3 into 5 is valid; silently folding a range overflow into a wrapped number is not.

The reference optimizer attempts successful constant computations but leaves failing operations in place so their runtime errors remain observable at the same operator. It does not claim that every theoretically possible optimization is implemented. Since the baseline has no variables, successful whole expressions can often fold to one literal, making it a clear teaching demonstration rather than a realistic optimizing compiler.

Algebraic rewrites require stronger reasoning. Replacing `(1/0)*0` by 0 would remove a required error. In a floating-point language, reassociating `(a+b)+c` as `a+(b+c)` can change rounding. An optimization is legal only under the specified domain and observations, not because the algebra is familiar over real numbers. [R14: optimizer support]

Use equivalence tests, but retain the proof obligation

Compare original tree evaluation, unoptimized VM and optimized VM on independently specified examples and generated bounded expressions. For valid expressions require equal values. For failures require the promised error category and source offset. A set of tests supports confidence but is not a proof over all accepted programs.

Explain the transformation locally: children are folded only into semantically equivalent subtrees; a parent is replaced only when its constant computation succeeds under the same arithmetic rule; otherwise its operation and location remain. That inductive argument is more informative than saying a thousand tests passed.

Gate C integrates the native simulation, resource manager, data structures, pathfinding or tooling and expression language. You must be able to explain the complete source-to-execution path and modify one new grammar case without copying the reference. The required C-language bridge follows this module for the hardware direction.

A fold that preserves an error

For `2+3*4`, fold 3*4 to 12 and then 2+12 to 14. Compilation can emit one PUSH 14 rather than five stack instructions. The original and optimized paths both produce 14 under the defined range.

For `(1/0)*0`, division fails. The optimizer leaves the division node rather than treating the entire product as zero. Execution still reports the division error at the slash, and the multiplication result is never produced.

For a finite-state literal recognizer, trace `007`: Start -> Digits -> Digits -> Digits, accept. Trace `7x`: Start -> Digits -> Dead, reject. Trace empty input: remain Start, reject. Numeric conversion is a separate subsequent check.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_15_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Conceptual folding rule:
fold children while preserving evaluation order;
if both operands are constant:
    try the language's checked arithmetic;
    if it succeeds, replace with a literal;
    if it fails, retain the operation and source location;
otherwise retain the operation;
```

Common failure modes

A lexical digit recognizer does not enforce numeric range. Undecidability is not a ban on proving a restricted evaluator terminates. A rewrite that removes a required error changes behavior. Passing equivalence tests is not a universal proof.

Practical assignment

Original requirement: Add constant folding to the toy compiler and test it against the unoptimized evaluator. Pass: include a finite-state recognizer, explain a termination restriction and document transformations deliberately excluded for semantic reasons.

1. Implement the finite-state decimal recognizer and separate lexical validity from numeric validity.
2. Add constant folding with explicit preservation of values and located runtime failures.
3. Compare original, bytecode and folded execution on golden and generated inputs.
4. Write a termination argument for the restricted language and list excluded transformations with semantic reasons.

Required cases: empty digits rejected; 007 accepted; 7x rejected; 2+3*4 folds to 14; (1/0)*0* retains division error; range overflow preserved as error; left-before-right error location retained; optimized and unoptimized valid results agree.

Evidence to submit: State-transition table, optimizer rules, equivalence tests, preserved-failure examples and restricted-termination argument.

Independent transfer: A proposed rewrite replaces `x-x` with zero after the language adds effectful function calls. Explain which assumptions would be required before approving it.

Assigned primary reading: R14: optimizer support; R06: computation models and proof limitations.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does undecidability mean no individual program can be proven to terminate?
2. Why can floating-point reassociation change a result?
3. What is the primary correctness condition for an optimization?
4. Why is replacing `(1/0)*0` by 0 invalid in this language?
5. What structural facts let this tiny evaluator terminate without a general halting solver?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.